

# Module 3 — Retrieval-Augmented Generation

## LLM, Prompt, RAG & Agents IA

Bassem Ben Hamed & Nesrine Trabelsi

Bootcamp

Avril 2026

## P.1 — Principes & Basic RAG

- Pourquoi RAG, pourquoi maintenant
- Définition et workflow
- RAG vs fine-tuning vs long-context
- Loading : LangChain loaders
- Chunking + Contextual Retrieval
- Embeddings : Bi-encoder, MTEB
- Vector stores : HNSW vs IVF
- Retrieval hybride (BM25 + RRF)
- Reranking : Cross-encoder
- Vigilance prod

## P.2 — Vector RAG

- Pipeline détaillé
- Forces, limites, cas d'échec

## P.3 — Advanced RAG

- Lost in the Middle
- Query Rewriting & Multi-Query
- HyDE
- Récap techniques avancées

## P.4 — Évaluation & Observabilité

- RAGas : 4 métriques
- LLM-as-a-judge & golden set
- LangSmith

## P.5 — Graph RAG (*bonus*)

- Motivation
- Extraction, graphe, communautés
- GraphRAG, Neo4j, LangChain
- Forces, coûts, maintenance

## P.6 — Comparaison & Agentic RAG

- Vector vs Graph + arbre décision
- Approches hybrides
- Anatomie d'un Agent IA
- Agent vs Agentic Workflow
- ReAct pattern
- 6 Agentic design patterns
- Adaptive RAG (grade docs/answer)
- Use case AI Research Assistant

## P.7 — Préparation TP 3

- Corpus + structure A/B/C/D
- Livrables & critères

# Objectifs d'apprentissage

À l'issue de ce module, vous serez capable de :

- 1 **Expliquer** pourquoi et quand ancrer un LLM sur des données externes plutôt que le fine-tuner.
- 2 **Concevoir** un pipeline **Basic RAG** complet : loading → chunking → embeddings → retrieval hybride → reranking → génération.
- 3 **Mettre en œuvre** les techniques d'**Advanced RAG** (Query Rewriting, Multi-Query, HyDE, lost-in-the-middle, Contextual Retrieval).
- 4 **Évaluer** avec **RAGas** (Faithfulness, Relevancy, Context Precision/Recall) et tracer avec **LangSmith**.
- 5 **Comprendre** l'apport du **Graph RAG** pour le multi-hop et le relationnel (*bonus*).
- 6 **Choisir** entre Basic, Advanced, Graph ou Hybride selon la question.
- 7 **Positionner** Agentic RAG comme orchestration dynamique — pont vers le Jour 3 (LangChain + LangGraph + n8n).

# Partie 1

Principes du RAG : de la connaissance paramétrique aux réponses ancrées

# Pourquoi les LLM seuls ne suffisent pas

## Limites structurelles du LLM seul

- **Knowledge cutoff** : pas les dernières publications, pas votre doc interne.
- **Hallucinations** : invente un chiffre, une référence, une fonction d'API.
- **Pas d'accès** aux données privées (wiki, KB, code source).
- **Pas de traçabilité** : d'où vient la réponse ?

## Conséquences opérationnelles

- Perte de confiance des utilisateurs.
- Risques conformité / RGPD selon le secteur.
- Coût du fine-tuning à chaque évolution.
- Adaptation lente à un savoir qui bouge.

## Question déclencheur

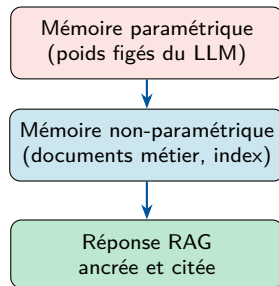
« Quelle architecture utilise GPT-5 ? » — un LLM seul répondra **plausiblement**, pas **exactement** (modèle sorti après son cutoff).

# Qu'est-ce que le RAG ?

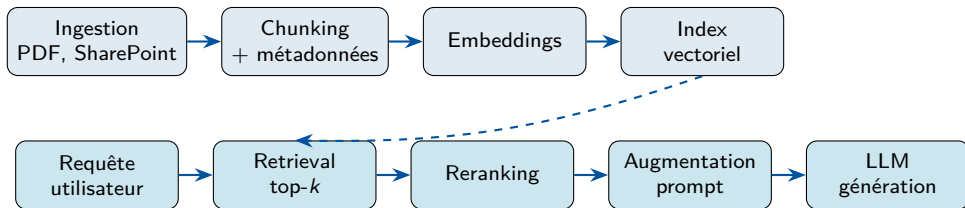
## Définition

Le **RAG** (Retrieval-Augmented Generation) combine une couche de **recherche d'information** avec un LLM : les réponses sont générées à partir de **preuves récupérées** plutôt que de la seule mémoire paramétrique.

- Flux : **Récupérer** → **Augmenter** → **Générer**.
- Deux types de mémoire :
  - **Paramétrique** — poids du modèle.
  - **Non-paramétrique** — documents, index, graphe.
- Gains : **factualité**, **fraîcheur**, **traçabilité**.



# Workflow RAG bout-en-bout



## Phase offline (indexation)

Préparer une fois — rafraîchir périodiquement.

## Phase online (requête)

À chaque appel utilisateur — latence critique.

*Principe : la qualité de la réponse est bornée par la qualité du retrieval.*

# RAG vs fine-tuning vs long-context

| Critère                | RAG                | Fine-tuning       | Long-context     |
|------------------------|--------------------|-------------------|------------------|
| Connaissances fraîches | ✓ Ré-indexation    | ✗ Ré-entraînement | ✓ Dans le prompt |
| Traçabilité sources    | ✓ Citations        | ✗ Opaque          | ✓ Visible        |
| Style / comportement   | ● Limité           | ✓ Natif           | ● Via prompt     |
| Coût à l'usage         | Moyen              | Faible/requête    | ✗ Très élevé     |
| Corpus > 100k docs     | ✓ Conçu pour       | ● Partiel         | ✗ Impossible     |
| Gouvernance données    | ✓ Droits sur index | ✗ Figé            | ✓ À la requête   |

## Règle de choix

- **RAG** pour **connaissances** qui bougent (procédures, FAQ, produits).
- **Fine-tuning** pour **comportement** stable (ton, style, format de sortie).
- **Combinaison** en production : fine-tuning comportement + RAG données.



# Cas d'usage RAG en entreprise

## Productivité interne

- Q&R sur la documentation technique / wiki / Confluence.
- Onboarding : assistant nouveaux collaborateurs.
- Recherche dans les procédures et guides internes.
- Synthèse de rapports, comptes-rendus, threads Slack/Teams.

## Recherche & support

- **AI Research Assistant** : papiers arXiv, blogs, biblio.
- Support N1 : ticket-to-answer sur knowledge base.
- Code search : assistant sur monorepo / docs API.
- Veille : digestion automatique de sources hétérogènes.

## Points sensibles à anticiper

Données privées (PII) — propagation des droits d'accès dans le retrieval — résidence des embeddings (UE / US / on-prem) — versionnage et fraîcheur du corpus.

# Ingestion : sources, nettoyage, métadonnées

## Sources typiques

- **Wiki / Confluence / SharePoint** : doc, FAQ, procédures.
- **Bases SQL / NoSQL** : référentiels métier.
- **PDF (papiers, contrats, rapports)** : OCR si scan.
- **Web** : papiers arXiv, blogs, sites publics.

## Métadonnées critiques

- `source_id`, `doc_type`, `date_maj`
- `niveau_confidentialite`, `departement`
- `version`, `auteur`, `url_source`

## Pièges fréquents

- En-têtes / pieds de page répétés qui polluent les chunks.
- Tableaux extraits en texte plat (perte de structure).
- PDF scannés sans OCR → chunks vides.
- Doublons (même procédure en v1, v2, v3).
- Absence de `date_maj` → impossible de filtrer les obsolètes.

# Loading : catalogue des LangChain loaders

## Format roi : Markdown

Le Markdown est *LLM-ready* : préserve la hiérarchie sémantique (titres, listes, tableaux) réexploitable pour le **chunking structurel** et les **métadonnées**. **Règle d'or** : convertir les sources en Markdown propre à l'**ingestion**.

| Source              | Loader recommandé                    | Notes                     |
|---------------------|--------------------------------------|---------------------------|
| PDF simple          | PyPDFLoader                          | Rapide, page par page     |
| PDF complexe        | DoclingLoader (IBM),<br>Unstructured | Multi-colonnes, scan, OCR |
| PDF → Markdown      | Marker                               | MD propre                 |
| Web (HTML)          | WebBaseLoader                        | BeautifulSoup             |
| Web → Markdown      | Firecrawl, Jina AI Reader            | Anti-bot, MD structuré    |
| Office (DOCX, PPTX) | UnstructuredLoader                   | Conversion homogène       |

## Ouverture — Multimodal RAG

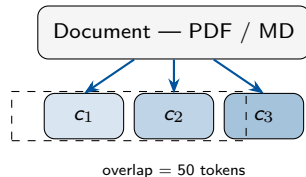
Les PDF de recherche / rapports techniques contiennent figures et tableaux. Les approches ColPali, Qwen2-VL, LayoutLMv3 embeddent les pages **comme images** en complément du texte.

# Chunking : le levier qualité n°1

- **Trop petit** (100 tokens) : contexte fragmenté.
- **Trop grand** (4000 tokens) : retrieval bruité.
- **Cible** : chunks cohérents et autonomes.

## Stratégies (LangChain + Chonkie)

- **Fixe / récursif** : RecursiveCharacterTextSplitter (défaut).
- **Structure-aware** : MarkdownHeaderTextSplitter respecte les titres.
- **Sémantique** : coupure aux frontières de sujet (Chonkie).
- **Agentique** : un LLM décide où couper (Chonkie).
- **Parent-child** : chunk petit indexé, parent retourné au LLM.



## Règle d'or

Ajuster taille + overlap **avec des métriques**, pas à l'intuition.

# Contextual Retrieval (Anthropic, 2024)

## Problème

Un chunk extrait isolément perd souvent son contexte global.

*Exemple* : « Les performances ont augmenté de 12 % sur le benchmark. » — mais quel modèle, quel benchmark, quelle version ?

## Recette

Avant l'embedding, demander à un LLM de **préfixer chaque chunk** d'un court résumé contextuel (1–2 phrases) sur la section parente.

Le préfixe est concaténé avant l'embedding *et* avant l'indexation BM25.

## Coût / bénéfice

- **Coût** : 1 appel LLM par chunk (offline, faisable).
- **Gain Anthropic** : –49 % sur le taux d'échec retrieval.
- –67 % combiné avec un reranker.

*À activer pour les corpus où les chunks sont ambigus hors-contexte (rapports, contrats, transcripts).*

# Embeddings : Bi-encoder et métriques de similarité

## Bi-encoder (utilisé en retrieval)

Le **même modèle** encode *indépendamment* :

chunk  $\rightarrow f(\text{chunk}) = \mathbf{c} \in \mathbb{R}^d$  (offline)

requête  $\rightarrow f(q) = \mathbf{q} \in \mathbb{R}^d$  (online)

puis  $\text{sim}(\mathbf{q}, \mathbf{c}) \rightarrow \text{top-}k$ .

## Choix du modèle — MTEB Benchmark

Comparer les modèles sur le **MTEB Leaderboard** (HF) avant tout choix : 56 datasets, 8 catégories.

## Trois métriques

- Cosinus :  $\cos(\mathbf{q}, \mathbf{c}) = \frac{\mathbf{q} \cdot \mathbf{c}}{\|\mathbf{q}\| \|\mathbf{c}\|}$
- Dot product :  $\mathbf{q} \cdot \mathbf{c}$  (sensible à la magnitude)
- L2 :  $\|\mathbf{q} - \mathbf{c}\|_2$

Si  $\|\mathbf{v}\| = 1$  : **cosinus** = **dot product** et  $L_2^2 = 2(1 - \cos)$ .

## Critères de choix

- Multilingue FR/AR.
- Dimension  $d + \text{max\_input\_tokens}$  ( $\geq \text{chunk\_size}$ , sinon troncature !).
- Latence vs qualité.
- Local (Ollama, HF) vs API.

# Embeddings : modèles courants et contraintes

| Modèle                          | $d$  | max tok. | Multilingue   | Notes                        |
|---------------------------------|------|----------|---------------|------------------------------|
| all-MiniLM-L6-v2                | 384  | 256      | ✗             | Léger, rapide, défaut        |
| all-mpnet-base-v2               | 768  | 384      | ✗             | +qualité, +lent              |
| multilingual-e5-large           | 1024 | 512      | ✓ 100 langues | FR/AR de bonne qualité       |
| BAAI/bge-m3                     | 1024 | 8192     | ✓             | Long contexte + multi-lingue |
| text-embedding-3-large (OpenAI) | 3072 | 8191     | ✓             | Top qualité, payant          |
| Azure OpenAI (idem ci-dessus)   | —    | —        | ✓             | Résidence UE                 |

## Astuce production

**L2-normaliser** les vecteurs ( $\|\mathbf{v}\| = 1$ ) → une seule métrique implémentée pour trois interprétations équivalentes.

## Cohérence indexation ↔ requête

Le **même modèle** doit être utilisé à l'indexation et à la requête. Changer de modèle d'embedding → **ré-indexer tout le corpus**.

# Vector stores : panorama

| Solution            | Forces                           | Limites                          | Adapté à              |
|---------------------|----------------------------------|----------------------------------|-----------------------|
| Azure AI Search     | Natif M365, hybride, Entra ID    | Coût, verrou fournisseur         | Stack Microsoft       |
| FAISS (Meta)        | Gratuit, rapide, local           | Pas de métadonnées avancées      | POC, notebooks TP 3   |
| Chroma              | Léger, Python-first, OSS         | Pas orienté prod scale           | TP 3, prototypes      |
| Pinecone / Weaviate | Managé, scalable, filtres riches | SaaS externe → résidence données | Prod cloud-agnostique |
| pgvector (Postgres) | SQL + vecteurs, transactionnel   | Latence à grande échelle         | Équipes DBA Postgres  |

## Capacités critiques à vérifier

- Filtrage par métadonnées (departement = "Conformite").
- Recherche **hybride** (BM25 + vectoriel), upsert incrémental.
- Isolation multi-tenant (utilisateurs, équipes, projets).



# Index ANN : HNSW vs IVF

## Pourquoi un index ANN ?

Recherche **exhaustive** :  $\mathcal{O}(N \cdot d)$  par requête — impraticable au-delà du million de vecteurs. Les **Approximate Nearest Neighbor** sacrifient un peu de recall pour des ordres de grandeur en latence.

| Approche               | HNSW (graphe)                           | IVF (clustering)                                                        |
|------------------------|-----------------------------------------|-------------------------------------------------------------------------|
| Principe               | Graphe multi-couches <i>small world</i> | $k$ -means $\rightarrow$ centroïdes ; on cherche dans $nprobe$ clusters |
| Hyperparams            | $M$ , $ef\_construction$ , $ef\_search$ | $nlist$ ( $\approx \sqrt{N}$ ), $nprobe$                                |
| Complexité             | $\mathcal{O}(\log N)$                   | $\mathcal{O}(\sqrt{N})$                                                 |
| Recall typique         | 95–99 %                                 | 90–95 %                                                                 |
| Empreinte mémoire      | Lourde (graphe stocké)                  | Légère                                                                  |
| Insertion incrémentale | Rapide                                  | Coûteuse (re-clustering)                                                |

## Règle pratique

Démarrer en **HNSW** (défaut Chroma / FAISS / Qdrant). Passer à **IVF + PQ** uniquement si la mémoire devient un problème ( $\geq 100M$  vecteurs).

# Retrieval : sparse, dense, hybride

| Approche                 | Principe                                        | Forces                                     | Faiblesses                       |
|--------------------------|-------------------------------------------------|--------------------------------------------|----------------------------------|
| Sparse (BM25)            | Correspondance de mots-clés, pondération TF-IDF | Exactes : codes produits, IBAN, références | Manque les paraphrases           |
| Dense (embeddings)       | Similarité cosinus de vecteurs                  | Paraphrases, synonymes, sens               | Peut manquer un token rare exact |
| Hybride (BM25 + vecteur) | Fusion des rangs (RRF, pondéré)                 | Meilleur recall, robuste                   | Plus complexe à régler           |

## Exemple : recherche hybride

Question : « Comment configurer le **retry\_on\_timeout** dans gRPC client v1.58 ».

Dense capte « configurer  $\approx$  paramétrer » ; BM25 verrouille le nom exact du flag et la version.

## RRF (Reciprocal Rank Fusion)

$$\text{RRF}(d) = \sum_{r \in \text{rankers}} \frac{1}{k + \text{rang}_r(d)}$$

$k \approx 60$ . **Robuste** : ignore les scores absolus (incomparables) et n'utilise que les rangs.

## Rappel BM25

$$\text{BM25}(D, Q) = \sum_i \text{IDF}(q_i) \cdot \frac{f(q_i, D)(k_1 + 1)}{f(q_i, D) + k_1(1 - b + b|D|/\bar{D})}$$

# Reranking : Bi-encoder vs Cross-encoder

## Distinction fondamentale

**Bi-encoder** (retrieval) :  $\text{chunk} \rightarrow [\text{Enc}] \rightarrow \mathbf{c}$ ,  $\text{query} \rightarrow [\text{Enc}] \rightarrow \mathbf{q}$ ,  $\text{score} = \cos(\mathbf{q}, \mathbf{c})$ .

**Cross-encoder** (reranking) :  $[q; \text{chunk}] \rightarrow [\text{Enc} + \text{classifieur}] \rightarrow \text{score} \in \mathbb{R}$ .

| Critère            | Bi-encoder                      | Cross-encoder              |
|--------------------|---------------------------------|----------------------------|
| Attention croisée  | ✗ Aucune                        | ✓ Pleine                   |
| Précision          | Bonne                           | Très haute                 |
| Pré-calcul chunks  | ✓ Une seule fois                | ✗ Impossible               |
| Complexité requête | $\mathcal{O}(\log N)$ via index | $\mathcal{O}(k)$ candidats |
| Usage              | Retrieval (top-50)              | Reranking (top-3 à 5)      |

## Modèles courants

- cross-encoder/ms-marco-MiniLM-L-6-v2 (TP)
- BAAI/bge-reranker-v2-m3 (open, multilingue)
- Cohere Rerank 3 (API)
- mxbai-rerank-large-v1

## Impact mesuré

+ **10 à 30 %** de **faithfulness** sans changer le LLM, pour quelques dizaines de ms ajoutées.

# Augmentation : le prompt template RAG

SYSTEME :

Tu es un assistant technique. Reponds UNIQUEMENT a partir du contexte fourni.  
Si l'information n'y figure pas, dis : "information non disponible dans la base documentaire".

CONTEXTE :

[chunk 1 - doc\_id: transformers\_paper, chunk\_id: 12]  
Self-attention computes Query, Key, Value matrices from the input.  
The attention output is  $\text{softmax}(QK^T / \sqrt{d_k}) * V$ , allowing  
the model to attend to all tokens simultaneously...

[chunk 2 - doc\_id: bert\_paper, chunk\_id: 3]  
BERT uses a stack of bidirectional Transformer encoders  
trained with masked language modeling...

QUESTION : Comment fonctionne le mecanisme de self-attention dans BERT ?

INSTRUCTIONS :

- Cite tes sources sous la forme [doc\_id:chunk\_id]
- Separe les faits (prouves) de l'inference
- Refuse si preuves insuffisantes

# Évaluation d'un système RAG : métriques clés

| Couche     | Métriques                                    | Interprétation                                                           |
|------------|----------------------------------------------|--------------------------------------------------------------------------|
| Retrieval  | Recall@k, Hit Rate, nDCG, MRR                | Les chunks pertinents sont-ils trouvés et bien classés ?                 |
| Génération | Faithfulness, Answer Relevance               | La réponse est-elle étayée par le contexte ? Répond-elle à la question ? |
| Contexte   | Context Precision, Context Recall            | Les chunks récupérés sont-ils utilisés et suffisants ?                   |
| Système    | Latence p50/p95, coût/requête, taux de repli | Le système est-il fiable et économiquement viable ?                      |

## Outillage

- **RAGAS** : métriques LLM-as-judge.
- **LangSmith** : traces + évaluation.
- **Azure AI Foundry** : évaluation native M365.

## Golden set

Construire 20–50 questions de référence avec réponses de confiance — **socle obligatoire** avant toute mise en production.

# Faithfulness : retrieval bon mais génération infidèle

## Scénario

Question : « Quel est le `max_tokens` par défaut pour ChatOpenAI dans LangChain v0.2 ? »

Retrieval retourne le bon chunk (doc API : la valeur par défaut est `None` = pas de limite côté SDK).

## Réponse infidèle

Le LLM dit : « La valeur par défaut est **4096**, **configurable via le constructeur** ».

Le **4096** et la formulation **configurable** ne sont pas dans le contexte → hallucination malgré bon retrieval.

## Contre-mesures

- Instruction ferme : « uniquement à partir du contexte ».
- Exiger des citations `[doc_id:chunk_id]`.
- Temperature basse (0–0.3).
- Vérification post-génération : chaque chiffre présent dans un chunk ?
- Politique de repli : « information non disponible ».

*La faithfulness se mesure, ne se suppose pas.*

# Points de vigilance en production

## Sécurité & gouvernance

- Propager les droits d'accès (SSO / IAM) au moment du retrieval.
- Isoler par tenant / projet / utilisateur.
- PII : pseudonymisation avant embedding si possible.
- Audit : log chaque requête + chunks retournés.

## Coûts & fraîcheur

- Embedding : coût one-shot à l'ingestion.
- Requête : LLM de génération >> embedding.
- Cache (requêtes fréquentes, embeddings).
- Ré-indexation incrémentale : détecter les mises à jour des sources (webhook ou scan planifié).
- Versionner le corpus (qui a changé quand).

## Modes d'échec typiques

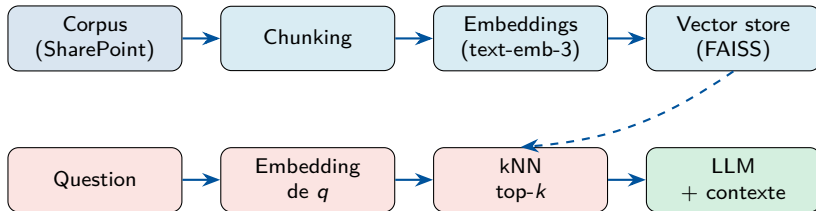
Échec retrieval → pollution du contexte → inversion de classement → ignorance du contexte → synthèse non étayée. **Ordre de débogage : retrieval → reranking → prompt → génération.**

# Partie 2

Vector RAG : pipeline, forces et limites



# Vector RAG : pipeline détaillé



- Indexation one-shot à gauche (offline) + ré-indexation incrémentale.
- À chaque requête (online) : embedding de  $q$  → recherche kNN → LLM avec contexte augmenté.

## Ce que Vector RAG fait très bien

- Questions **factuelles isolées** : « Quelle est la signature de la fonction  $f()$  ? »
- **Recherche sémantique** : paraphrases, synonymes, reformulations.
- **Corpus large** : passe l'échelle (millions de chunks).
- **Mise en place rapide** : outillage mature (LangChain, LlamaIndex, Chroma, Pinecone).
- **Résumé de document** unique.

## Pourquoi c'est la valeur par défaut

- 70–80 % des cas d'usage sont des Q&R factuelles sur un document.
- Coût maîtrisable, latence faible ( $< 1$  s).
- Écosystème open-source mature, hosting flexible.
- Montée en compétence en quelques jours pour une équipe DATA.

# Vector RAG : limites

## Relations implicites perdues

Les embeddings capturent la **proximité de sens**, pas les **relations structurelles** entre entités. Exemple : « Quelles méthodes de fine-tuning améliorent BERT » — les relations « dérive de », « améliore » sont diluées dans le texte.

## Raisonnement multi-hop faible

Vector RAG récupère des chunks **indépendants**. Il ne sait pas « suivre une chaîne » :  $A \rightarrow B \rightarrow C$ . Pour reconstituer une chaîne, il faut que **tous les chunks** soient récupérés **simultanément** — rarement le cas.

## Autres limites

- **Opacité du retrieval** : pourquoi ce chunk a été choisi ? Difficile à expliquer à un auditeur.
- **Requêtes globales** : « Quels sont *tous* les produits avec un taux  $> X$  ? » — Vector voit par fenêtres, pas en vue d'ensemble.
- **Contexte fragmenté** : une info partagée entre 5 documents  $\rightarrow$  top- $k$  risque d'en rater.

# Cas concret où Vector RAG échoue

## Question utilisateur (revue de littérature)

« Quels papiers proposant un mécanisme d'attention **sub-quadratique** citent **Linformer** ET ont été **appliqués aux séquences génomiques** ? »

## Pourquoi Vector RAG galère

- 3 hops : papier → technique, papier → citations, papier → domaine.
- Infos dans plusieurs sources : abstract, biblio, sections d'application.
- Le top- $k$  capte des chunks **isolés**, pas la **jointure**.
- Aucun chunk « contient » la réponse — elle doit être **construite**.

## Ce qu'il faut à la place

- Un **graphe** reliant papiers, techniques, citations, domaines.
- Un retrieval qui **traverse** les arêtes.
- OU un **agent** qui décompose et interroge plusieurs sources.

→ *argument Graph RAG et Agentic RAG.*

# Partie 3

Advanced RAG : du prototype à l'outil robuste

# Limites du Basic RAG — ce qu'on attaque

| Problème                        | Symptôme                                               | Solution                |
|---------------------------------|--------------------------------------------------------|-------------------------|
| <i>Lost in the Middle</i>       | L'info au milieu du contexte est ignorée               | Reorder + reranking     |
| Bruit dans les chunks récupérés | Documents quasi-pertinents qui dérivent la réponse     | Reranking, filtres méta |
| Requêtes courtes / imparfaites  | « C'est quoi LoRA ? » — peu de matière à embedder      | Query Rewriting         |
| Vocabulary mismatch (synonymes) | La requête et les chunks utilisent des mots différents | Multi-Query, HyDE       |
| Mauvais ordonnancement          | Le bon chunk est top-20, pas top-3                     | Cross-encoder rerank    |
| Mots-clés exacts manqués        | Acronymes, codes produits, IBAN                        | Hybride BM25 + dense    |

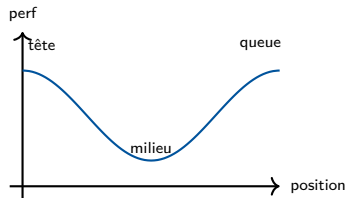
## Méthode

Pour chaque problème : (1) symptôme observable, (2) levier ciblé, (3) gain mesurable sur le **golden set**.

# Lost in the Middle (Liu et al., 2023)

## Phénomène

Quand on injecte beaucoup de chunks dans le prompt, les LLMs **utilisent moins l'information située au milieu** du contexte. La performance forme une courbe en **U** : les chunks *en début* et *en fin* sont privilégiés.



## Conséquences pratiques

- Top-10 n'est pas toujours mieux que top-3 — on dilue l'attention.
- L'**ordre de présentation** compte → mettre les meilleurs aux extrémités.
- Argument fort pour avoir un reranker (récupérer 20, garder 3, trier).

## Pattern « lost-in-the-middle proof »

Best en **tête**, 2nd best en **queue**, le reste au milieu.

# Query Rewriting et Multi-Query

## Query Rewriting

Demander au LLM de **réécrire la requête** avant retrieval pour :

- retirer le bruit conversationnel ;
- ajouter les termes techniques implicites ;
- normaliser le français approximatif.

Ex : « et HNSW ? »

→ « Comment fonctionne HNSW pour la recherche ANN ? »

## Multi-Query Retrieval

Le LLM génère  $N$  **reformulations** (synonymes, sous-questions). On retrouve pour chacune et on **fusionne avec RRF**.

- Compense le *vocabulary mismatch*.
- Récupère des chunks inatteignables seule.
- Coût :  $N$  retrievers (parallélisables).

## Coût / bénéfice

**Query Rewriting** : 1 appel LLM, gain  $\approx +5$  à  $+15$  % Recall. activer sur requêtes courtes et conversationnelles.

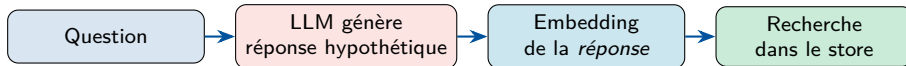
**Multi-Query** :  $1 + N$  appels, gain  $\approx +10$  à  $+20$  %. À



# HyDE — Hypothetical Document Embeddings (Gao et al., 2022)

## Idée

La requête et le document vivent dans des **espaces sémantiques différents** (question vs réponse). On comble le gap en demandant au LLM de **générer une réponse hypothétique**, puis on **embedde cette réponse fictive** plutôt que la question.



## Pourquoi ça marche

La réponse hypothétique **ressemble structurellement** aux documents pertinents (même registre, même vocabulaire technique) → similarité plus élevée que la question brute.

## Limite

Le LLM peut **halluciner** dans la réponse hypothétique — peu importe, on n'utilise que son **embedding**, pas son contenu. Risque résiduel sur sujets très pointus inconnus du LLM.

# Récap des techniques avancées — coût, gain, quand activer

| Technique                  | Coût               | Quand activer                      | Gain typique             |
|----------------------------|--------------------|------------------------------------|--------------------------|
| Hybride BM25 + dense + RRF | Aucun              | <b>Toujours</b> en prod            | +5 à +15 % Recall        |
| Re-ranking cross-encoder   | Modèle dédié       | <b>Toujours</b>                    | +10 à +30 % Faithfulness |
| Query Rewriting            | 1 appel LLM        | Requêtes courtes / conv.           | +5 à +15 % Recall        |
| Multi-Query                | 1 + $N$ appels     | Vocabulary mismatch                | +10 à +20 % Recall       |
| HyDE                       | 1 appel LLM        | Requêtes très courtes / abstraites | +5 à +25 % Recall        |
| Lost-in-the-middle reorder | Aucun              | $\text{top}_k \geq 5$              | +3 à +10 % Faithfulness  |
| Contextual Retrieval       | $N$ appels offline | Chunks ambigus hors-contexte       | -49 % échecs             |

## Heuristique de combinaison

1. Hybride BM25 + dense + RRF (incontournable)
2. Re-ranking cross-encoder (incontournable)
3. Multi-Query *ou* HyDE selon les requêtes
4. Lost-in-the-middle reorder en sortie.

# Partie 4

RAGas, LLM-as-a-judge, LangSmith

# RAGas — les 4 métriques clés

Le RAG est itératif : Construire → Mesurer → Analyser → Améliorer

Sans mesure, on optimise à l'aveugle. Avec RAGas, on identifie quel levier (chunking, embedding, retrieval, prompt, LLM) tirer en priorité.

| Métrique                 | Question posée                         | Formule (intuition)                                        |
|--------------------------|----------------------------------------|------------------------------------------------------------|
| <b>Context Precision</b> | Chunks récupérés pertinents ?          | # chunks pertinents / # chunks récupérés                   |
| <b>Context Recall</b>    | A-t-on tout récupéré ?                 | # infos réponse présentes / # infos réponse                |
| <b>Faithfulness</b>      | Réponse ancrée (pas d'hallucination) ? | # affirmations supportées / # affirmations totales         |
| <b>Answer Relevancy</b>  | Réponse pertinente ?                   | $\cos(\text{question}, \text{questions reverse-générées})$ |

## Lecture

Retrieval (Precision/Recall) et Génération (Faithfulness/Relevancy) se diagnostiquent **séparément** — ce qui guide où corriger.

## Principe

Toutes les métriques RAGas utilisent un **LLM comme évaluateur**. Exemple Faithfulness :

- 1 Le LLM décompose la réponse en **affirmations atomiques**.
- 2 Pour chaque affirmation : « est-elle supportée par le contexte ? »
- 3 Faithfulness = ratio des affirmations supportées.

## Bonnes pratiques

- LLM-juge **plus puissant** que le LLM-générateur (sinon plafond artificiel).
- **Golden set** 50–200 paires (question, réponse, sources).
- Calibrer le juge sur 30 cas annotés humainement.

## Frameworks alternatifs

- **TruLens** : observabilité + éval, dashboard.
- **DeepEval** : pytest-style, intégration CI/CD.
- **Phoenix** (Arize) : tracing + éval local.
- **Azure AI Foundry** : éval native M365.

# Observabilité avec LangSmith

## Pourquoi tracer ?

En RAG production, **80 % des bugs sont des problèmes de retrieval invisibles** : un mauvais chunk, un prompt trop long, une latence anormale dans une étape précise.

## Ce qu'une trace LangSmith contient

```
rag_pipeline      2.3s    1240 tok
  retrieve_hybrid  50ms
    retrieve_dense  30ms
    retrieve_sparse 18ms
  rerank          120ms
  generate_answer 2.1s    1240 tok
    ollama.chat
      prompt:      "... "
      completion: "... "
```

## Activation (2 lignes)

```
os.environ["LANGSMITH_TRACING"]="true"
os.environ["LANGSMITH_API_KEY"]="lsv_..."
```

Tout LangChain est tracé automatiquement. Pour les fonctions Python pures : décorateur `@traceable`.

## Alternatives stack

OpenTelemetry + Grafana / Tempo / Jaeger pour rester self-hosted ; Azure Application Insights pour la stack Microsoft.

## Partie 5 (*bonus*)

Graph RAG : relations explicites et raisonnement multi-hop

# Graph RAG : motivation

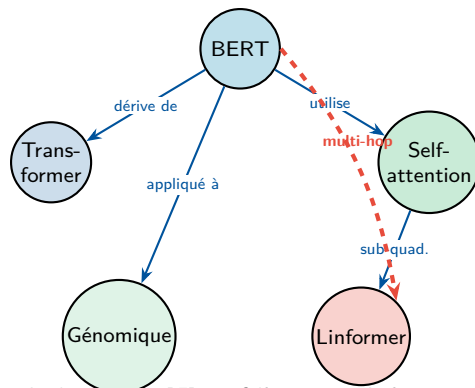
## Limite fondamentale du chunk-based

Le retrieval par chunks **ne voit pas** les relations explicites entre entités distribuées dans plusieurs documents.

## Intuition Graph RAG

Représenter le corpus comme un **graphe typé** plutôt qu'une liste plate de chunks :

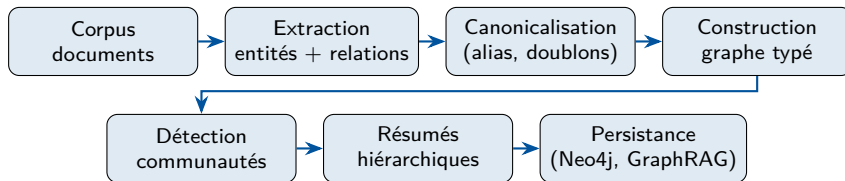
- Nœuds = entités (concepts, modèles, méthodes, auteurs).
- Arêtes = relations (« cite », « dérive de », « utilise »).
- Retrieval → **traversée** du graphe.



Le chemin rouge « BERT → Self-attention → Linformer » n'est PAS dans un seul chunk : il faut traverser le graphe.



# Pipeline de construction Graph RAG



## Point critique

La qualité du graphe dépend directement de la **précision de l'extraction** d'entités/rerelations et de la **conception du schéma** (types de nœuds et d'arêtes).

# Extraction d'entités et relations : en pratique

Prompt systeme :

Tu es un extracteur d'entites et relations pour un corpus technique.  
Extrais sous forme JSON strict.

Document :

"BERT (Devlin et al., 2018) repose sur un stack d'encodeurs Transformer bidirectionnels et est entraine par masked language modeling. Le papier cite Vaswani et al. (2017) pour l'architecture Transformer."

Sortie attendue :

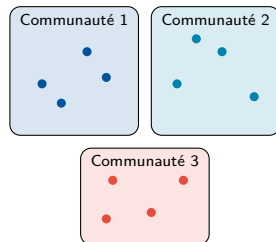
```
{
  "entites": [
    {"id": "BERT",      "type": "MODELE",   "annee": 2018},
    {"id": "Transformer", "type": "ARCHITECTURE"},
    {"id": "MLM",        "type": "METHODE"},
    {"id": "Vaswani 2017", "type": "PAPIER"}
  ],
  "relations": [
    {"src": "BERT", "rel": "DERIVE_DE", "dst": "Transformer"},
    {"src": "BERT", "rel": "UTILISE", "dst": "MLM"},
    {"src": "BERT", "rel": "CITE", "dst": "Vaswani 2017"}
  ]
}
```

# Microsoft GraphRAG : approche par communautés

## Idée clé

Après extraction, le graphe est **segmenté en communautés** (Leiden) — sous-graphes densément connectés.

- Chaque communauté reçoit un **résumé LLM** (local).
- Agrégation en **résumés globaux** (sous-domaine, corpus complet).
- Retrieval : route la question vers le **bon niveau**.



## Deux modes de requête

- **Local** : question sur une entité précise.
- **Global** : synthèse sur tout le corpus.

# Stratégies de retrieval Graph RAG

## Retrieval local

- Identifier les **entités ancrs** dans la question.
- Traverser les voisins à 1–2 hops.
- Récupérer les chaînes de preuves courtes.
- Idéal : « Quels papiers BERT cite-t-il directement ? »

## Retrieval global

- Partir des **résumés de communautés**.
- Fusion top-down des réponses partielles.
- Idéal : « Quelles familles d'architectures concurrentes au Transformer apparaissent dans le corpus ? »

## Hybride Graph + Vector

La pratique recommandée : combiner **retrieval vectoriel** (pour le contexte textuel riche) et **traversée graphe** (pour les relations structurelles), puis fusionner au niveau du prompt.

# Outils Graph RAG

| Outil                     | Positionnement                                                   | Cas d'usage type                        |
|---------------------------|------------------------------------------------------------------|-----------------------------------------|
| Microsoft GraphRAG        | Framework open-source complet (extraction, communautés, résumés) | Corpus non-structuré, requêtes globales |
| Neo4j + LangChain         | Base graphe mature + retrievers graphes                          | Schéma connu, requêtes Cypher           |
| LlamaIndex KG             | Knowledge Graph Index, intégré RAG                               | Prototypage rapide, pipelines Python    |
| Azure Cosmos DB (Gremlin) | Graphe managé Azure                                              | Intégration cloud Microsoft             |

## Choix recommandé pour le TP 3

**Microsoft GraphRAG** (extraction LLM-based + communautés) sur le même corpus que Vector RAG, pour une comparaison directe. Neo4j en option si le schéma est déjà défini.

# Graph RAG : forces et coûts

## Forces

- **Multi-hop natif** : traverser des relations.
- **Requêtes globales** : synthèse sur tout le corpus.
- **Explicabilité** : chemin graphe traçable.
- **Évolutivité sémantique** : ajout de relations sans réindexer tout.

## Coûts & contraintes

- **Extraction coûteuse** : 1 appel LLM / document.
- **Maintenance** continue du graphe.
- **Erreurs d'extraction** qui se propagent.
- **Complexité ops** : graphe + vecteur + orchestration.

## Règle de décision

Utiliser Graph RAG **quand la structure relationnelle est centrale** aux questions — pas par défaut.

# Partie 6

Vector vs Graph, Hybride, et teaser Agentic RAG

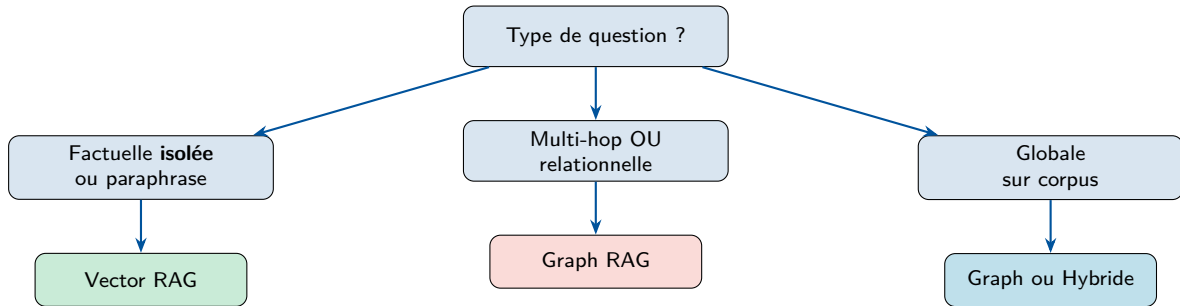
# Vector RAG vs Graph RAG : tableau comparatif

| Critère                    | Vector RAG | Graph RAG  | Hybride  |
|----------------------------|------------|------------|----------|
| Factuelle isolée           | ✓          | ●          | ✓        |
| Résumé d'un document       | ✓          | ●          | ✓        |
| Multi-hop / relationnelle  | ✗          | ✓          | ✓        |
| Requête globale sur corpus | ✗          | ✓          | ✓        |
| Paraphrases / synonymes    | ✓          | ●          | ✓        |
| Corpus très large          | ✓          | ●          | ●        |
| Coût d'ingestion           | Moyen      | ✗ Élevé    | Élevé    |
| Maintenance                | Faible     | ✗ Continue | Continue |
| Explicabilité              | Moyenne    | ✓ Traçable | Élevée   |
| Temps de mise en œuvre     | Jours      | Semaines   | Semaines |

✓ = fort, ● = moyen, ✗ = faible.



# Arbre de décision : quel RAG pour quelle question ?



## Heuristique verbale

- Questions en « qui », « quoi », « combien » isolés → **Vector**.
- « Comment X est lié à Y », « tous les ... », « via Z » → **Graph**.
- En cas de doute ou d'audience mixte → **Hybride**.

# Approches hybrides : combiner le meilleur des deux

## Pattern 1 : Vector puis Graph

- 1 Vector RAG retourne top- $k$  chunks.
- 2 Extraction des **entités** dans ces chunks.
- 3 Traversée du graphe autour de ces entités (+1 hop).
- 4 Fusion des deux contextes dans le prompt.

## Pattern 2 : Graph puis Vector

- 1 Identifier les **entités ancrées** de la question.
- 2 Traversée du graphe pour trouver les entités liées.
- 3 Vector RAG filtré sur les documents associés à ces entités.
- 4 LLM avec contexte enrichi.

## Quand choisir l'hybride ?

Dès que vos utilisateurs posent **deux types de questions différents** sur le même corpus — ce qui est très fréquent (questions opérationnelles + questions analytiques).

# Teaser Agentic RAG : pont vers le Jour 3

## De RAG statique à RAG orchestré

Jusqu'ici, le RAG est **réactif** : on l'interroge, il répond. Avec **Agentic RAG**, un **agent LLM** décide **quand**, **quoi** et **combien de fois** récupérer — et peut **reformuler**, **auto-évaluer**, **rechercher sur le web** si besoin.

## Ce que l'agent apporte

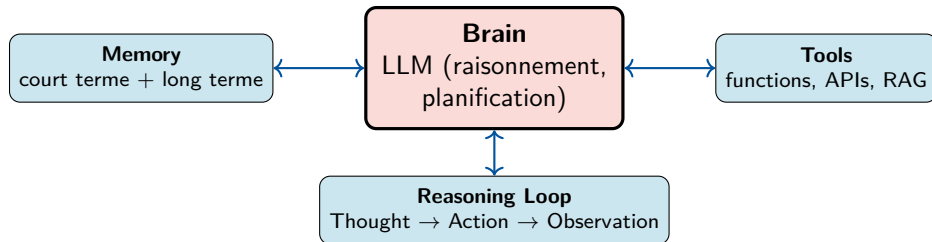
- **Décision** : récupérer ou répondre direct ?
- **Routage** : Vector, Graph, SQL, web, arXiv ?
- **Décomposition** : éclater une question.
- **Auto-critique** : preuves suffisantes ? (*grade docs / grade answer*)
- **Itération** : relancer, élargir, fallback web.

## Cap pédagogique — Jour 3

Demain, vous construisez un **AI Research Assistant** :

- En **Python avec LangChain** (ReAct) puis **LangGraph** (workflow contrôlé).
- En **no-code avec n8n** (AI Agent node + tools).

# Anatomie d'un Agent IA



## Composants

- **Brain** : LLM, system prompt, paramètres.
- **Memory** : conversationnelle (turn-by-turn), épisodique (sessions), sémantique (faits).
- **Tools** : RAG, recherche web (Tavily), APIs, code execution, n8n triggers.

## Schéma de raisonnement

- L'agent **ne sait pas** d'avance combien de tours.
- À chaque tour : observe le contexte, planifie, décide d'un outil ou de répondre.
- Stoppe quand le LLM produit une *Final Answer* (ou `max_iter`).

# Agent vs Agentic Workflow

| Critère            | Agent (autonome)                              | Agentic Workflow                                                 |
|--------------------|-----------------------------------------------|------------------------------------------------------------------|
| Contrôle du flux   | Le <b>LLM</b> décide à chaque tour quoi faire | Le <b>développeur</b> fixe les nœuds, le LLM décide aux branches |
| Prévisibilité      | Faible — comportement émergent                | Élevée — chemins explicites                                      |
| Outil typique      | LangChain <code>create_react_agent</code>     | <b>LangGraph</b> , <code>n8n</code>                              |
| Robustesse en prod | Variable                                      | ✓ Meilleure (loops, retries, branches gérés)                     |
| Évaluation / debug | Difficile (chaînes longues)                   | ✓ Inspection nœud par nœud                                       |
| Bon pour           | Exploration, prototype, tâches ouvertes       | Production, processus métier balisé                              |

## Pratique recommandée

Démarrer en **Agent ReAct** (rapide à itérer) → figer ensuite les chemins critiques en **LangGraph** / `n8n` (audit, SLA, observabilité). Anthropic, 2024 : « *Build the simplest workflow that works ; only graduate to agents when needed.* »

# Le pattern ReAct (Reasoning + Acting)

## Idée (Yao et al., 2022)

Le LLM alterne explicitement **Thought** (réflexion en langage naturel) et **Action** (appel d'outil). Chaque action produit une **Observation** qui alimente le tour suivant.

Question: Combien de papiers citent "Linformer" ET sont appliqués à la génomique ?

Thought: Je dois chercher des papiers liés à Linformer dans le corpus, puis filtrer sur génomique.

Action: vector\_search("Linformer attention")

Observation: 4 papiers retournés.

Thought: Lesquels mentionnent la génomique ?

Action: vector\_search("genomic sequence model")

Observation: 2 papiers en commun.

Thought: J'ai assez d'evidence.

Final Answer: BERT-genome (2020) et BigBird-DNA (2021).

## Pourquoi ça marche

- Le **Thought** sert de **plan** et de **trace** d'audit.
- Le LLM peut **revenir en arrière** si l'observation infirme l'hypothèse.
- Réduit les hallucinations vs. LLM seul.

## Limite

Coûteux : N tours  $\Rightarrow$  N appels LLM. À encadrer par `max_iter` et un *stop* robuste.

# Agentic Design Patterns (Anthropic, 2024)

| Pattern                 | Principe                                                                           | Cas d'usage                                                                  |
|-------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| 1. Prompt chaining      | Décomposer une tâche en étapes séquentielles (output $n \rightarrow$ input $n+1$ ) | Génération de rapport : plan $\rightarrow$ rédaction $\rightarrow$ relecture |
| 2. Parallélisation      | Plusieurs LLMs traitent indépendamment, puis on agrège (vote, fusion)              | Ensemble de classifieurs, RAG multi-source                                   |
| 3. Routing              | Un classifieur LLM dispatche la requête vers le bon spécialiste                    | Vector vs Graph vs SQL vs Web                                                |
| 4. Reflection loop      | Le LLM critique sa propre sortie et la révisé (self-eval)                          | Code review, génération de spécifications                                    |
| 5. Orchestrator-workers | Un orchestrateur découpe et distribue à des sous-agents « workers » spécialisés    | Recherche multi-document, ingénierie complexe                                |
| 6. Tool use (ReAct)     | Le LLM <i>décide</i> d'appeler des outils en fonction du contexte                  | AI Research Assistant (notre use case)                                       |

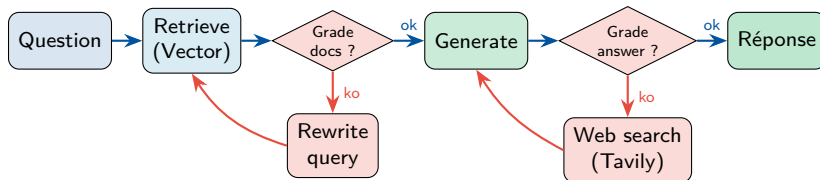
## Combinaison typique en production

**Routing** en entrée  $\rightarrow$  **Tool use** (RAG, web, APIs)  $\rightarrow$  **Reflection** en sortie  $\rightarrow$  **Prompt chaining** pour la synthèse finale.

# Adaptive RAG : grade docs, grade answer, web fallback

## Idée

L'agent ne fait pas confiance aveuglément à son retrieval. Il **évalue** chaque chunk récupéré, **évalue** sa propre réponse, et **bascule sur une recherche web** si nécessaire.



## Variantes connues

**Self-RAG** (Asai et al., 2023) ; **Corrective RAG / CRAG** (Yan et al., 2024) ; **Adaptive RAG** (Jeong et al., 2024). Toutes implémentables en **LangGraph**.



# Use case Jour 3 — AI Research Assistant

## Spec fonctionnelle

Un assistant qui répond à des questions de recherche en consultant :

- le **vector store** local (notre corpus indexé du TP 3) ;
- **Tavily** (Search API LLM-friendly, gratuit *tier*) pour le web ;
- un outil **arXiv** pour la recherche académique ;
- un outil **summarize** pour digérer les résultats.

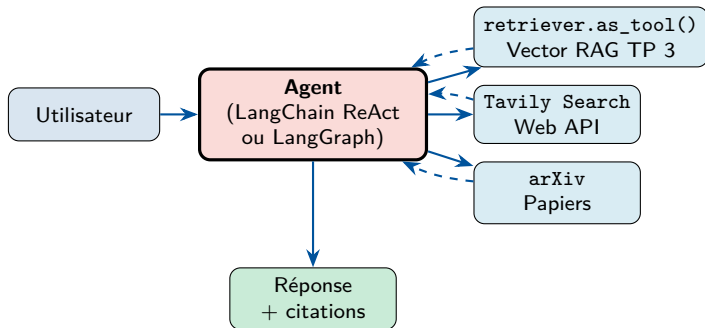
## Implémentations parallèles

- **Python + LangChain** :  
`create_react_agent, Tool, retriever.as_tool()`.
- **Python + LangGraph** : workflow Adaptive RAG (grade docs / grade answer / web fallback).
- **n8n no-code** : AI Agent node, tools déclarés en JSON, observabilité visuelle.

## Pourquoi ce double parcours ?

**LangChain/LangGraph** pour comprendre **ce qui se passe** sous le capot ; **n8n** pour montrer comment livrer un agent en **production légère** sans tout coder.

# Schéma d'orchestration — Agent « AI Research Assistant »



## Hors scope du TP 3

On reste aujourd'hui sur Basic + Advanced + Graph **statiques**. L'orchestration agent + outils + Adaptive RAG vient au **TP 4** (Jour 3) avec **LangChain** → **LangGraph** → **n8n**.

# Partie 7

TP 3 : pipeline RAG complet (Basic + Advanced + Eval) + Graph  
(bonus)

## Objectif

Pipeline RAG complet : **Basic RAG** (LangChain) → **Advanced RAG** (HyDE, Multi-Query, Lost-in-the-middle) → **Évaluation RAGas** ; bonus **Graph RAG**.

## Corpus fourni

- 4 docs Markdown sur l'IA : Transformer, RAG, fine-tuning, vector DBs.
- Page Wikipedia RAG (loader web).
- Format Markdown → **LLM-ready**.
- Métadonnées : `source`, `format`, `section`, `chunk_id`.

## Outillage (open-source)

- `langchain`, `langchain-community`
- `sentence-transformers` (Bi + Cross-enc)
- `chromadb` + `rank_bm25`
- `ollama` + `glm-4.7-flash` (local, gratuit)
- `ragas`, `langsmith` (optionnel)
- `networkx` (Graph RAG bonus)

# TP 3 — Structure des 3 heures

| Temps  | Partie                              | Livrables                                                                                                                                                        |
|--------|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 h 15 | <b>A — Basic RAG</b>                | Loaders LangChain + Splitters (Recursive*, MarkdownHeader*) + Contextual Retrieval + embeddings + Chroma + BM25 + RRF + cross-encoder rerank + génération Ollama |
| 45 min | <b>B — Advanced RAG</b>             | Lost-in-the-middle reorder + Query Rewriting + Multi-Query + HyDE, mesurés sur le golden set                                                                     |
| 30 min | <b>C — Eval &amp; Obs.</b>          | RAGas (Faithfulness, Relevancy, Precision, Recall) avec LLM-juge Ollama + cellule LangSmith                                                                      |
| 30 min | <b>D — Graph RAG (<i>bonus</i>)</b> | Extraction entités/rerelations + graphe NetworkX + retrieval local/global + comparaison Vector vs Graph                                                          |

## Les 5 questions de référence — adaptées à votre cas d'usage

- 1 Q1 — factuelle isolée (ex. « Signature de la fonction  $f$ ? »).
- 2 Q2 — factuelle avec paraphrase (même info, mots différents).
- 3 Q3 — relationnelle à 2 hops (concept A → méthode → B).
- 4 Q4 — requête globale (synthèse sur le corpus).
- 5 Q5 — multi-hop 3+ (entité → relation → sous-domaine → application).

# TP 3 — Livrables et critères d'évaluation

## Livrables par participant

- Notebook versionné (parties A + B + C exécutées, D si le temps le permet).
- Résultats **RAGas** sur leur propre corpus (3–5 questions golden set).
- Mini rapport (5–10 lignes) : cas d'usage métier, approche choisie (Basic / Advanced / Graph), métriques observées.
- **Au moins une technique Advanced testée** (Multi-Query *ou* HyDE *ou* Contextual Retrieval).
- **Choix motivé** d'une approche pour leur cas d'usage métier réel.

## Critères d'évaluation

- Qualité du pipeline (chunking, embeddings, retrieval, rerank).
- Pertinence des choix Advanced (Multi-Query / HyDE selon requêtes).
- Sérieux de l'évaluation (golden set représentatif, métriques RAGas mesurées).
- Justification du choix final.
- Prise en compte sécurité / gouvernance des données.

# Récapitulatif : 4 messages clés

## 1. Retrieval > Génération

La qualité de la réponse est **bornée** par la qualité du retrieval. Investir d'abord dans le **chunking**, la **recherche hybride** et le **reranking** — avant de changer de LLM.

## 2. Basic RAG, puis Advanced quand utile

Basic RAG (hybride + reranker) couvre 70–80 % des besoins. Activer **Multi-Query** / **HyDE** / **Contextual Retrieval** *ciblément* sur les pain-points mesurés du golden set.

## 3. Vector par défaut, Graph quand nécessaire

**Graph RAG** n'est rentable que lorsque la **structure relationnelle** est centrale aux questions. L'**hybride** (Vector + Graph) est souvent le bon compromis en production.

## 4. Pas de production sans mesure

**Golden set** + **RAGas** (Faithfulness, Relevancy, Precision, Recall) + traces **LangSmith** + seuils go/no-go **avant** mise en production. La démo qui marche ne suffit pas.

# Transition vers le Jour 3

## Ce que vous avez construit aujourd'hui

- Un pipeline **Basic RAG** complet (LangChain + hybride + cross-encoder).
- Des techniques **Advanced RAG** (Query Rewriting, Multi-Query, HyDE, Contextual Retrieval).
- Une **évaluation RAGas** mesurée et tracée (LangSmith).
- Un prototype **Graph RAG** (*bonus*) et un cadre de comparaison.

## Ce que vous ajoutez demain (Jour 3) — AI Research Assistant

- Agent **ReAct en Python** avec LangChain ; `retriever.as_tool()` sur le RAG du TP 3.
- **Adaptive RAG** en **LangGraph** : *grade docs* → *rewrite* ; *grade answer* → *web fallback* (Tavily) + arXiv.
- Même use case en **no-code via n8n** (AI Agent node + tools).
- Anatomie d'un agent + **6 design patterns** (chaining, parallélisation, routing, reflection, orchestrator, tool use).

**Aujourd'hui : RAG statique — Demain : RAG piloté par un agent**



# Références et lectures recommandées

## Papiers fondateurs

- Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, 2020.
- Edge et al. (Microsoft Research), *From Local to Global: A Graph RAG Approach*, 2024.
- Asai et al., *Self-RAG*, 2023.

## Advanced RAG

- Liu et al., *Lost in the Middle*, 2023.
- Gao et al., *Precise Zero-Shot Dense Retrieval without Relevance Labels* (HyDE), 2022.
- Anthropic, *Contextual Retrieval*, 2024 (blog).

## Outils & documentation

- LangChain — [python.langchain.com](https://python.langchain.com)
- Chonkie — [github.com/chonkie-inc/chonkie](https://github.com/chonkie-inc/chonkie)
- RAGas — [docs.ragas.io](https://docs.ragas.io)
- LangSmith — [smith.langchain.com](https://smith.langchain.com)
- MTEB Leaderboard — [huggingface.co/spaces/mteb/leaderboard](https://huggingface.co/spaces/mteb/leaderboard)
- Microsoft GraphRAG — [microsoft.github.io/graphrag](https://microsoft.github.io/graphrag)

## Frameworks d'éval alternatifs

TruLens, DeepEval, Phoenix (Arize), Azure AI Foundry.

# Questions ?

Prêts pour le TP 3 ?